



Advanced Programming

Chapter 4: Servlets

Outlines

- Servlet Overview
- Servlet Architecture
- Servlet Task
- Servlet API
- Servlet Lifecycle
- Servlet Page Redirection
- Servlet Database Access

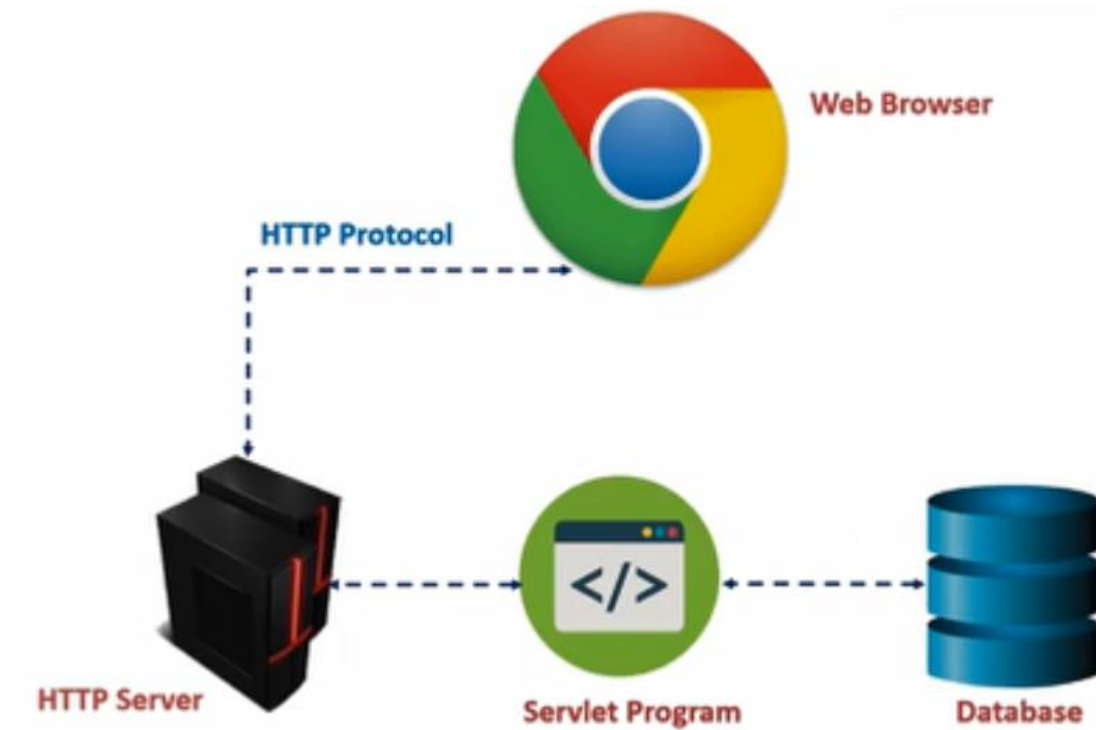
Servlet overview

- Servlets are **programs that run on a Web** or Application server.
- It act as a **middle layer** between a requests coming from a Web browser or other HTTP client and databases.
- Servlet is used to:
 - Collect input from users through web page forms
 - Present records from a database or another source and
 - Create web pages dynamically

The advantages of Servlets over CGI

- CGI(Common Gateway Interface) is a **standard for writing programs** that can interact through a web server with a client running a web browser.
- Servlets offer several advantages in comparison with the CGI.
 - **Performance** is significantly better
 - **Execute** within the address space of a Web server
 - **Platform-independent** because they are written in Java
 - **Secured**
 - **Communicate** with applets, databases, or other software

Servlets Architecture

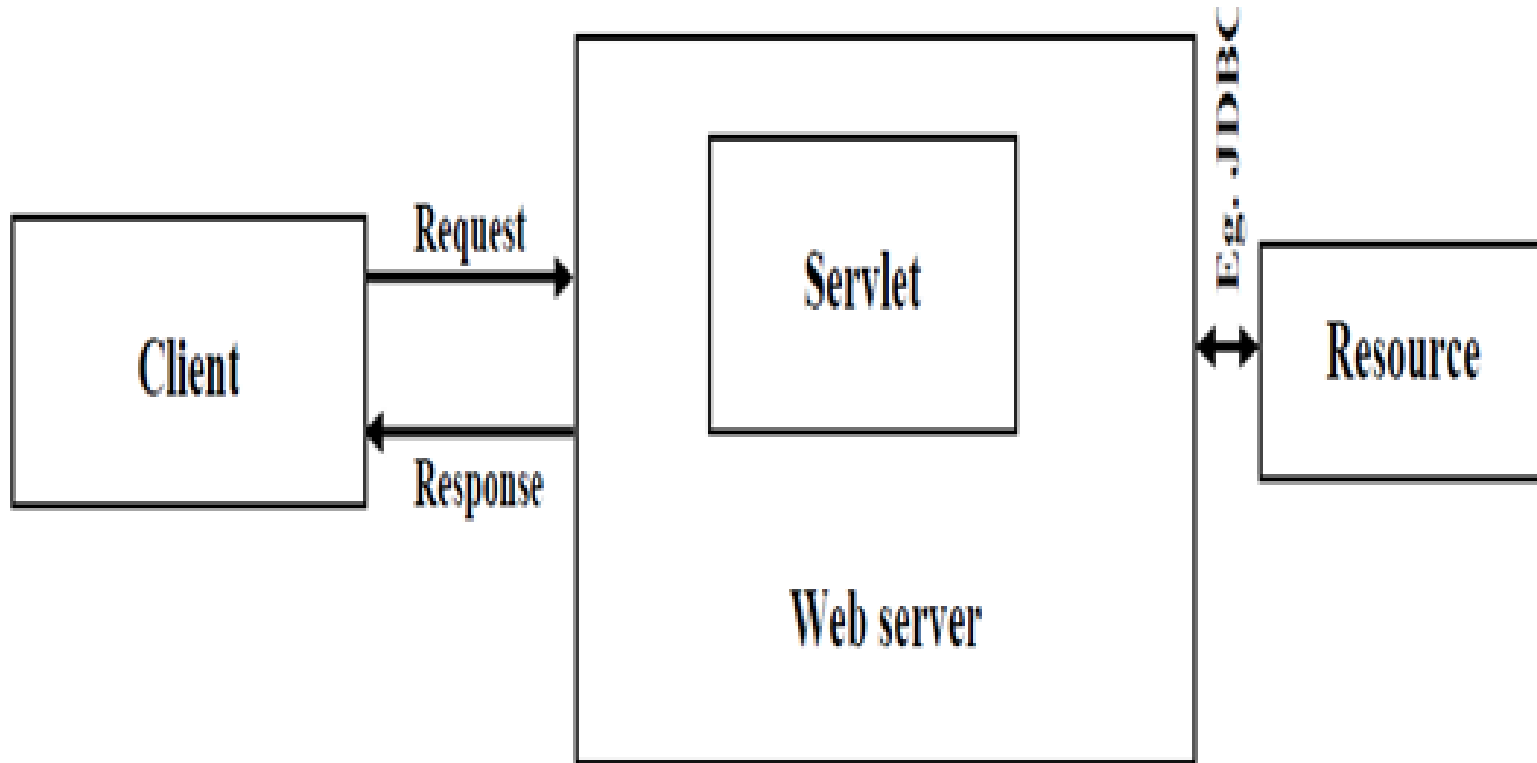


Servlets Tasks

- Read the explicit data sent by the clients (browsers).
- Read the implicit HTTP request data sent by the clients (browsers)
- Process the data and generate the results
- Send the explicit data (i.e., the document) to the clients (browsers).
- Send the implicit HTTP response to the clients (browsers).

Servlets Tasks...

The following diagram shows High level client-to-servlet process flow



The Java Servlet API

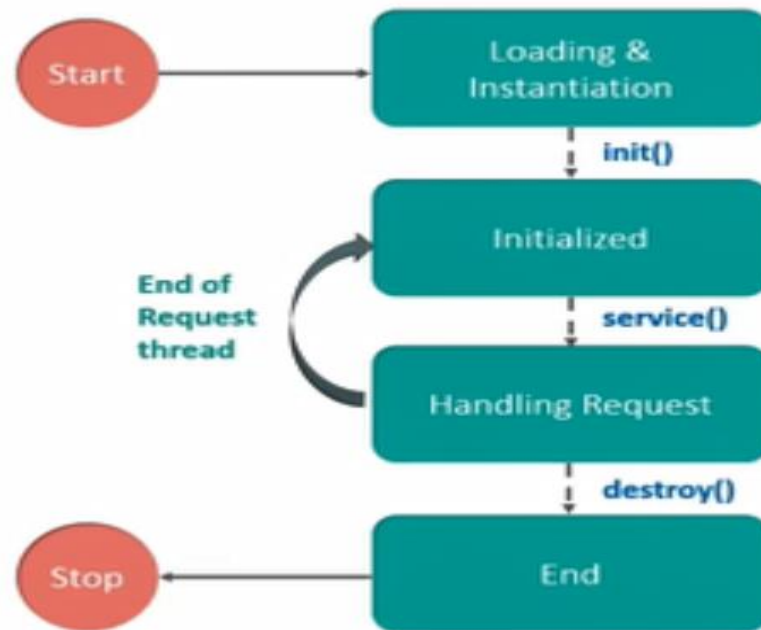
- Two things are required to develop servlets:
 - Servlet **API class** files, which are used for **compiling** the servlets and
 - Servlet **container** such as a web server, which is used for **running** the servlets.
- Servlet API is a set of **Java classes** which define a standard **interface** between a **Web client** and a **Web servlet**

Servlet API....

- The API is composed of two packages:
 - **javax.servlet** and **javax.servlet.http**
- **javax.servlet** Contains classes to support generic protocol-independent servlets. e.g: **GenericServlet**
- **javax.servlet** defines the methods which servlets must implement, like **service()** methods
- **javax.servlet.http** package support for the **HTTP protocol**.
- **HttpServlet.HttpServlet** provides methods for the processing of **HTTP requests** such as **GET** (**doGet** method) and **POST** (**doPost** method).

Servlets Life Cycle

- The servlet is initialized by calling the **init()** method.
- The servlet calls **service()** method to process a client's request.
- The servlet is terminated by calling the **destroy()** method.
- Finally, servlet is **garbage** collected by the garbage collector of the JVM.



Life Cycle...

init() Method:

- It is called only when the servlet is created.

```
public void init() throws ServletException {  
  
    // Initialization code...  
  
}
```

Life Cycle...

The service() Method:

- Main method to perform the actual task
- Servlet container (i.e. web server) calls the **service()** method to handle requests coming from the client(browsers)
- **doGet()** and **doPost()** methods are used with in each service request.
- Checks the HTTP request type (GET, POST, PUT, DELETE, etc.)

```
public void service(ServletRequest request,  
    ServletResponse response)  
    throws ServletException, IOException{  
}
```

Life Cycle...

doGet() Method:

- A GET request results from normal request for a URL or from an HTML form with no METHOD specified.
- Data is sent in header.
- Limited data can be sent
- Not secured

```
public void doGet(HttpServletRequest request,
HttpServletRequest response)
throws ServletException, IOException {
// Servlet code
}
```

Life Cycle...

doPost() Method:

- A POST request results from an HTML form that specifically lists POST as the METHOD.
- Data is sent in request body
- Large amount of data can be sent
- It is secured

```
public void doPost(HttpServletRequest request,  
HttpServletResponse response)  
throws ServletException, IOException {  
    // Servlet code  
}
```

Life Cycle...

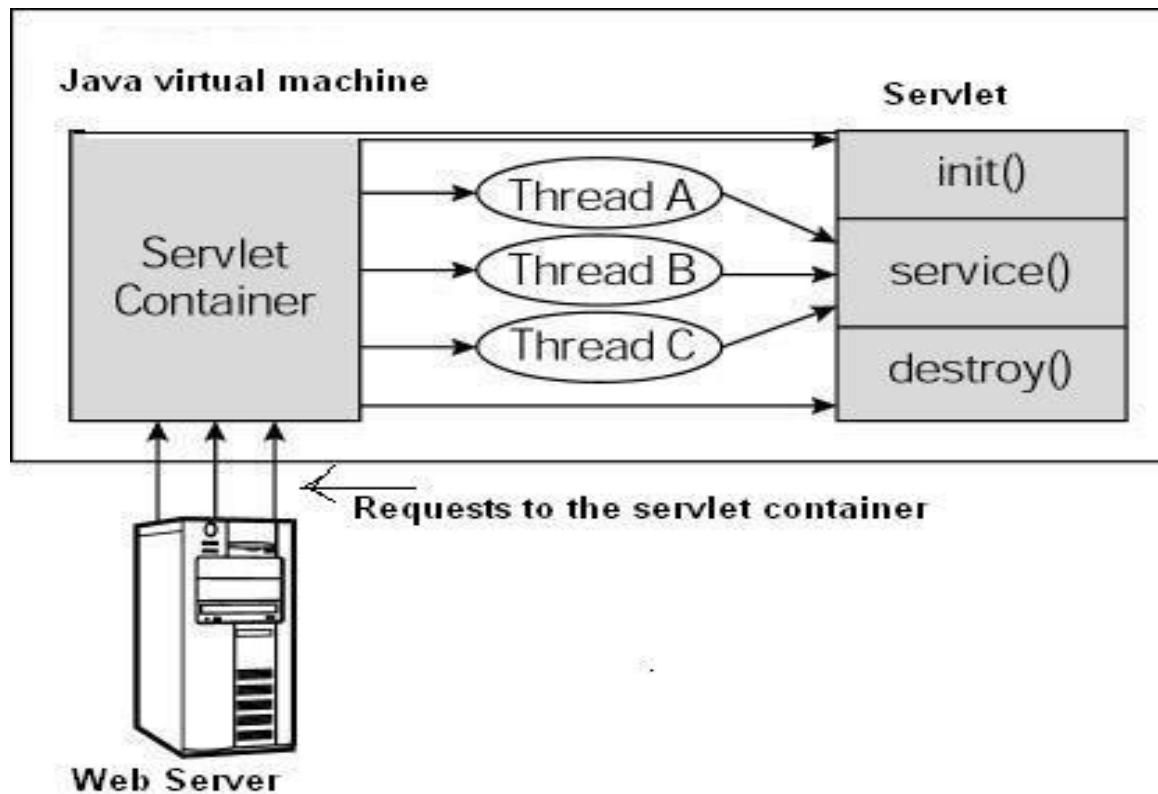
The destroy() Method:

- It is called only once at the **end** of the life cycle of a servlet.
- It causes the servlet to close **database connections**, **halt background threads**, **write cookie lists** or **hit counts to disk**, and perform other such **cleanup** activities.

```
public void destroy()
{
    // Finalization code...
}
```

Life Cycle...

- The following figure depicts a typical servlet life-cycle scenario.



Servlet example

```
//servlet source code that displays Hello
World.
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
// Extend HttpServlet class
public class HelloWorld extends HttpServlet {
    private String message;
    public void init() throws ServletException {
        // Do required initialization
        message = "Hello World";
    }
}
```

Servlet example...

```
public void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
    // Set response content type
    response.setContentType("text/html");
    // Actual logic goes here.
    PrintWriter out = response.getWriter();
    out.println("<h1>" + message + "</h1>");
}
public void destroy() {
    // do nothing.
}
}
```

Servlets Page Redirection

- Page redirection is a technique where the client is sent to a new location other than requested.
- It is generally used when a document moves to a new location or may be because of load balancing.
- **sendRedirect()** method is used for redirecting a request to another page.

```
public void HttpServletResponse.sendRedirect(String  
location) throws IOException
```

Page Redirection...

- This example shows how a servlet performs page redirection to another location.

```
import java.io.*;
import java.sql.Date;
import java.util.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class PageRedirect extends HttpServlet {
    public void doGet(HttpServletRequest request,
        HttpServletResponse response) throws ServletException, IOException
    {
        // Set response content type
        response.setContentType("text/html");
        // New location to be redirected
        String site = new String("http://www.photofuntoos.com");
        response.setStatus(response.SC_MOVED_TEMPORARILY);
        response.setHeader("Location", site);
    }
}
```

Page Redirection...

- Now let us compile above servlet and create the following entries in web.xml.
- **web.xml** defines mappings between URL paths and the **servlets** that handle requests with those paths.
- The **web** server **uses** this configuration to identify the **servlet** to handle a given request and call the class method

```
....  
<servlet>  
    <servlet-name>PageRedirect</servlet-name>  
    <servlet-class>PageRedirect</servlet-class>  
</servlet>  
  
<servlet-mapping>  
    <servlet-name>PageRedirect</servlet-name>  
    <url-pattern>/PageRedirect</url-pattern>  
</servlet-mapping>  
....
```

Servlets Database Access

➤ To start with interfacing Java Servlet Program with JDBC Connection:

1. Proper JDBC Environment should set-up along with database creation.
2. To do so, download the mysql-connector.jar file from the internet,
3. As it is downloaded, move the jar file to the apache-tomcat server folder,
4. Place the file in lib folder present in the apache-tomcat directory.
5. Start the basic concept of interfacing.

Database Access...

➤ To start with the basic concept of interfacing:

- Step 1: Creation of Database and Table in MySQL
 - As soon as jar file is placed in the folder, create a database and table in MySQL.
- Step 2: Creation of Java Servlet program with JDBC Connection.
 - To create a JDBC Connection, the steps are:
 - » Import all the packages
 - » Register the JDBC Driver
 - » Open a connection
 - » Execute the query, and retrieve the result
 - » Clean up the JDBC Environment

Database Access...

- Here is an example which shows how to access TEST database using Servlet

```
// Loading required libraries
import java.io.*;

import java.util.*;

import javax.servlet.*;

import javax.servlet.http.*;

import java.sql.*;

public class DatabaseAccess extends HttpServlet{

    public void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

        // JDBC driver name and database URL
        static final String JDBC_DRIVER = "com.mysql.jdbc.Driver";
        static final String DB_URL="jdbc:mysql://localhost/TEST";

        // Database credentials
        static final String USER = "root";
        static final String PASS = "password";
```

Database Access...

```
// Set response content type
response.setContentType("text/html");
PrintWriter out = response.getWriter();
String title = "Database Result";

String docType = "<!doctype html public \"-//w3c//dtd html 4.0 \"
+ \"transitional//en\">\n";
out.println(docType +
    "<html>\n" +
    "<head><title>" + title + "</title></head>\n" +
    "<body bgcolor = \"#f0f0f0\">\n" +
    "<h1 align = \"center\">" + title + "</h1>\n");
try {
    // Register JDBC driver
    Class.forName("com.mysql.jdbc.Driver");
    // Open a connection
    Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
```

Database Access...

```
// Execute SQL query
Statement stmt = conn.createStatement();
String sql;
sql = "SELECT id, first, last, age FROM Employees";
ResultSet rs = stmt.executeQuery(sql);

// Extract data from result set
while(rs.next()){
    //Retrieve by column name
    int id  = rs.getInt("id");
    int age = rs.getInt("age");
    String first = rs.getString("first");
    String last = rs.getString("last");

    //Display values
    out.println("ID: " + id + "<br>");
    out.println(", Age: " + age + "<br>");
    out.println(", First: " + first + "<br>");
    out.println(", Last: " + last + "<br>");
}
out.println("</body></html>");
```

Database Access...

```
// Clean-up environment
rs.close();
stmt.close();
conn.close();
} catch(SQLException se) {
    //Handle errors for JDBC
    se.printStackTrace();
} catch(Exception e) {
    //Handle errors for Class.forName
    e.printStackTrace();
} finally {
    //finally block used to close resources
    try {
        if(stmt!=null)
            stmt.close();
    } catch(SQLException se2) {
    } // nothing we can do
    try {
        if(conn!=null)
            conn.close();
    } catch(SQLException se) {
        se.printStackTrace();
    } //end finally try
} //end try
}
}
```

Database Access...

- Now let us compile above servlet and create following entries in web.xml.

....

```
<servlet>
    <servlet-name>DatabaseAccess</servlet-name>
    <servlet-class>DatabaseAccess</servlet-class>
</servlet>
```

```
<servlet-mapping>
    <servlet-name>DatabaseAccess</servlet-name>
    <url-pattern>/DatabaseAccess</url-pattern>
</servlet-mapping>
```

....

Questions?